Dining Philosophers Using Processes & System V Semaphores

Objective

Implement the Dining Philosophers problem using fork() and System V semaphores.

Problem Analysis

- Five philosophers and five forks.
- Each philosopher is a separate process.
- Each fork is represented by one semaphore.
- A philosopher must acquire both forks before eating.
- Deadlock must be avoided.

Deadlock Avoidance

- Even philosophers acquire LEFT then RIGHT fork.
- Odd philosophers acquire RIGHT then LEFT fork.
- Alternative: Allow only four philosophers to compete using an additional semaphore.

Algorithm

1. Create semaphore set using semget().

2. Initialize all semaphores to 1 using semctl().

3. Create five child processes using fork().

4. Each child repeats Think -> Hungry -> Acquire -> Eat -> Release.

5. Parent waits using wait().

6. Delete semaphore set using IPC_RMID.

System Calls

- fork()
- wait()
- semget()
- semop()
- semctl()

- sleep()
- exit()

Pseudo Code

Parent:
Create semaphores
fork five children
wait for children
remove semaphores

Child:
repeat 5 times
think
wait(fork1)
wait(fork2)
eat
signal(fork1)
signal(fork2)
exit

Sample Viva Questions

1. What is a semaphore?
2. Why use System V semaphores?
3. What causes deadlock?
4. How is deadlock avoided?
5. What is IPC_RMID?
6. What is starvation?
7. Why is wait() required?

Common Mistakes

- All processes acquiring left fork first.
- Not deleting semaphores.
- Not checking system call return values.
- Parent exiting before children.